Song Sorters
Course: COP3530
Project: Project 2
Group Name: Song Sorters
Members: Alwin Paul, Rafael Hitoshi Teixeira Oura & Ricardo Chiacchio Ribeiro

1. Introduction:

The aim of this project is to sort a given list of songs based on the number of views they have. We will look at how two different sorting algorithms (Merge Sort and Quick Sort) affect performance.

Each song will have a title and number of views. We will create two data sets of songs each containing 500 songs and 10000 songs.

This project was created in C++. This menu driven program creates song lists, sorts using the algorithms, displays the top songs and compares the runtimes. We also used the C++ std::sort to compare with.

2. Design & Implementation

Data Structure:

Within this project we have a struct called Song. This struct holds the following:

- title - This string holds the title of the song.
- viewCount - This integer holds the number of views.

The songs are stored within a vector. A vector was chosen due to allowing us to store a varying amount of songs and allowing us to access specific elements using indexes.

The generateSongs() function was used to generate the desired amount of songs. Each song will have a unique title such as 'Song 1’ and a random number of views between 0 and 1 Billion. As the random seed is set this will give the same data set for testing.

Merge Sort:

Merge Sort is one of the sorting algorithms used in the project. This uses a divide and conquer method.

It splits the list into smaller pieces until each piece contains one element. These pieces are then merged together in a sorted order. Within this project we are comparing based on the number of views so the song with the most views will be placed first.

As we merge the two sorted lists we use a temporary vector. If two songs have the same amount of views the left hand element will be chosen first. This means that Merge Sort is stable.

Merge Sort has a time complexity of O(n log n) for all cases. It has a space complexity of O(n) due to the temporary vector.

Quick Sort:

Quick Sort is the second sorting algorithm. Like Merge Sort this uses a divide and conquer method.

To determine the pivot we used the median of three methods. This looks at the first, middle and last elements of the list. We then used the Hoare partition method to move the higher viewed songs to the front and lower viewed songs to the end.

If two songs have the same number of views we will handle this within the sort. We then recursively sort the smaller partition. The larger partition is handled within a loop to avoid too much recursion.

Quick Sort has an average time complexity of O(n log n). However it has a worst case time complexity of $O(n^2)$. This can happen if the partitions are continually uneven. The recursion stack has an average space complexity of O(log n). As we are recursing on the smaller partition the space complexity will stay at O(log n) even when partitions are uneven. Quick Sort is not stable so songs with the same number of views may not stay in their original order.

Benchmark:

We used the C++ std::sort to compare against. This is not a substitute for the two custom sorts. Both Merge Sort and Quick Sort were created from scratch.

Each algorithm will receive their own copy of the original data set to ensure they do not get a sorted list. We will compare the results to ensure that the custom sorts are returning the songs in descending order of views and that all original song records are there.

Test Results:

We tested to ensure that our sorting algorithms were working correctly. This is important to ensure we have correctly sorted the songs without losing any records.

We tested a variety of scenarios:

- Empty list.
- One song.
- Sorted.
- Unsorted.
- Duplicate views.
- More songs.
- All songs retained.

The benchmark also compared to the reference sort to ensure that the custom sorts are returning the correct order. We also ensured that the original song titles and views were present.

Catch2 Results: [Number of tests passed] [Number of tests failed]

Runtime Comparison:

We used the  library in C++ to determine how long the sorting algorithms took to run. The timer started just before the sorting function and stopped just after.

The original data set is copied to create separate copies of the data set. This ensures we are timing the sorting and not the copying of the vectors.

We compared the Merge Sort, Quick Sort and std::sort with both data sets.

Runtime for 500 songs:

- Merge Sort: [Time taken] milliseconds.
- Quick Sort: [Time taken] milliseconds.
- std::sort: [Time taken] milliseconds.

Runtime for 10000 songs:

- Merge Sort: [Time taken] milliseconds.
- Quick Sort: [Time taken] milliseconds.
- std::sort: [Time taken] milliseconds.

After running the benchmarks we can see which algorithm ran fastest. The time taken varied depending on the data set size.

Looking at the time complexities we can see why this may have happened. Merge Sort will always run in O(n log n). Quick Sort will run on average in O(n log n). However this depends on how well the pivot splits the data. The median of three method reduces the chances of an uneven split.

We can now compare the theoretical results to the actual results.

3. Time Complexity of Key Functions:

- Generate Songs: O(n) time complexity and O(n) space complexity due to creating and storing the songs.
- Merge Sort: O(n log n) time complexity for best, average and worst cases. O(n) extra space for the temporary vector.
- Quick Sort: O(n log n) average time complexity. $O(n^2)$ worst case time complexity. Recursively sorting the smaller partition means that the recursion stack has an O(log n) space complexity.
- Benchmark: The time complexity of the sorting depends on the chosen algorithm. Additional checks are done after sorting.
- Top 10 Songs: A copy of the whole vector is made and the vector is sorted. This has an overall time complexity of O(n log n) due to sorting using either custom sort.

4. Modifications:

Ensuring that both algorithms sorted in the correct order. Both need to sort from most views to least views.

Ensuring that songs with the same amount of views were handled correctly. Merge Sort is stable and will retain the original order of songs with the same amount of views. Quick Sort is not stable and cannot guarantee this.

Ensuring that the benchmark was fair. Each algorithm will receive their own copy of the original unsorted vector.

During implementation we fixed issues with the Merge Sort merge and improved the benchmark. We now check that the songs are sorted in the correct order and that the original songs are unchanged.

In the original proposal we wanted to create a website. However we created a menu driven console application. This kept the focus on implementing and comparing the sorting algorithms.

5. Roles:

- Alwin Paul – Generate song data and support for two data sets.
- Rafael Hitoshi Teixeira Oura – Implement Merge Sort and tests.
- Ricardo Chiacchio Ribeiro – Implement Quick Sort and tests.
- All – Benchmark system, analysing results, writing report and creating demonstration video.

6. Reflection:

This project helped us to understand how two different sorting algorithms can be used to solve the same problem. Both Merge Sort and Quick Sort use the divide and conquer method but differ in how they are implemented.

Merge Sort has a consistent time complexity of $O(n \log n)$. However extra memory is required to merge the partitions. Quick Sort generally performs well but has a low space complexity as we are using a small amount of extra stack. However it has a worst case time complexity of $O(n^2)$.

This project helped us to understand the importance of testing. An algorithm may look to be working with a small list however there could be issues when using larger data sets or duplicate values. By comparing to a reference sort we can be confident our results are correct.

Overall this project has given us hands on experience of implementing sorting algorithms, using vectors and structs, testing C++ functions and measuring runtime.

7. Conclusion:

Song Sorters compares Merge Sort and Quick Sort. We looked at how they sort song lists based on their number of views. We looked at both data sets of 500 songs and 10000 songs to see how the algorithms perform as more data is added.

Theoretical analysis shows that Merge Sort will run in O(n log n) for all cases. Quick Sort will run on average in O(n log n) and worst case in $O(n^2)$. We can compare the algorithms practically using the benchmark results.

8. References:


1. University of Florida. COP3530 Sorting Algorithms Lecture.
2. C++ Reference. std::sort. https://en.cppreference.com/w/cpp/algorithm/sort
3. C++ Reference. std::chrono. https://en.cppreference.com/w/cpp/chrono


9. Links:

GitHub: https://github.com/Rafatoshi/Song-Sorter-COP3530

Video: [Video link]